

Hunting Linux Credential Access Attacks using Auditd and Wazuh

Project Overview

This project explores the practical implementation of **Linux auditd** for monitoring file activities and system calls, followed by integration with **Wazuh SIEM** for centralized log analysis and real-time security monitoring.

The project is divided into two major phases:

1. **Exploration of auditd** – Installation, rule creation, and validation of Linux audit logging.
2. **Integration with Wazuh** – Forwarding audit logs to Wazuh, creating detection logic, and validating alerts through simulated attack scenarios.

The objective of this project is to detect and hunt for **Linux credential access attacks**, monitor sensitive file activity, and generate actionable alerts for SOC analysts.



Author: Abdul Rehman

Cybersecurity | SOC | AWS Security Projects

[LinkedIn](#)

What is Auditd?

Auditd (Linux Audit Daemon) is a powerful native Linux framework used to:

- Monitor file activity (read/write/delete)
- Track system calls (execve, open, chmod, etc.)
- Capture user activity and privilege escalation attempts
- Provide forensic-level logging

It works at the **kernel level**, making it highly reliable for security monitoring and forensic investigation.

Why Use Auditd?

1 Monitoring File Activities

Auditd can track:

- Who accessed a file
- When it happened
- What command was used
- Under which privilege level

This is critical for monitoring:

- /etc/passwd
- /etc/shadow
- SSH keys
- Bash history files

2 Security Incident Investigation

Audit logs provide:

- Irrefutable forensic evidence
- Failed operations tracking
- Privilege abuse detection
- Timeline reconstruction during breach analysis

3 Regulatory Compliance

Many industries require:

- Detailed activity logs
- System call auditing
- Privilege tracking

Auditd helps organizations meet compliance requirements such as:

- ISO 27001
- PCI-DSS

- HIPAA
- Government security standards

Why Integrate Auditd with Wazuh?

Auditd alone generates logs.

Wazuh adds:

- ✓ **Real-time Alerting**
- ✓ **Custom Detection Rules**
- ✓ **MITRE ATT&CK Mapping**
- ✓ **Centralized Monitoring**
- ✓ **Dashboard Visualization**
- ✓ **Threat Hunting Capability**

Integration creates a powerful security monitoring framework for SOC environments.

Architecture Overview

Linux Host (Auditd Enabled)

↓

/var/log/audit/audit.log

↓

Wazuh Agent

↓

Wazuh Manager

↓

Custom Rules + CDB Lists

↓

Wazuh Dashboard (Detection & Analysis)

Tools Used

- Linux (Ubuntu)
- auditd
- Wazuh Agent
- Wazuh Manager
- Wazuh Dashboard

- Custom Wazuh Rules
- CDB Lists

Implementation Steps

◆ Step 1 – Auditd Installation

Auditd and required plugins were installed using:

```
sudo apt install auditd audispd-plugins
```

This installed:

- audit daemon
- audit utilities
- plugins for log processing

Purpose:

To enable kernel-level logging of file activity and system calls

```
root@cortex:/home/ubuntu# sudo apt -y install auditd audispd-plugins
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libauparse0t64
The following NEW packages will be installed:
  audispd-plugins auditd libauparse0t64
```

◆ Step 2 – Configuring Audit Rules

Custom audit rules were created inside:

/etc/audit/rules.d/audit.rules

Rules added to monitor:

- Execution of commands (execve)
- Access to /etc/passwd
- Access to /etc/shadow
- SSH directory access
- Bash history access

Example rule:

```
-a always,exit -F arch=b64 -S execve -k audit-wazuh-c
```

Purpose:

To monitor credential access and command execution activity.

```
## First rule - delete all
-D

## Increase the buffers to survive stress events.
## Make this bigger for busy systems
-b 8192

## This determine how long to wait in burst of events
--backlog_wait_time 60000

## Set failure mode to syslog
-f 1

-a exit,always -F arch=b64 -F auid!=-1 -F euid!=0 -S execve -k audit-wazuh-c
-a exit,always -F arch=b64 -F path=/etc/shadow -F auid!=-1 -F euid!=0 -F perm=r -k shadow_access
-a exit,always -F arch=b64 -F path=/etc/passwd -F auid!=-1 -F euid!=0 -F perm=r -k passwd_access
-a exit,always -F arch=b64 -F path=/home/ubuntu/.bash_history -F auid!=-1 -F euid!=0 -F perm=r -k history_access
-a exit,always -F arch=b64 -F dir=/home/ubuntu/.ssh/ -F auid!=-1 -F euid!=0 -F perm=r -k ssh_access
```

◆ Step 3 – Verifying Rule Activation

Rules were loaded using:

```
sudo auditctl -R /etc/audit/rules.d/audit.rules
```

```
sudo auditctl -l
```

Verification ensured:

- Rules were successfully loaded
- Kernel auditing was enabled
- Buffer limits were configured

```
root@cortex:/home/ubuntu# sudo auditctl -R /etc/audit/rules.d/audit.rules
No rules
enabled 1
failure 1
pid 10374
rate_limit 0
backlog_limit 8192
lost 0
backlog 3
backlog_wait_time 60000
backlog_wait_time_actual 0
enabled 1
failure 1
pid 10374
rate_limit 0
backlog_limit 8192
lost 0
backlog 3
backlog_wait_time 60000
backlog_wait_time_actual 0
enabled 1
failure 1
pid 10374
rate_limit 0
backlog_limit 8192
lost 0
backlog 4
backlog_wait_time 60000
backlog_wait_time_actual 0
```

◆ Step 4 – Testing Auditd Functionality

Test activity performed:

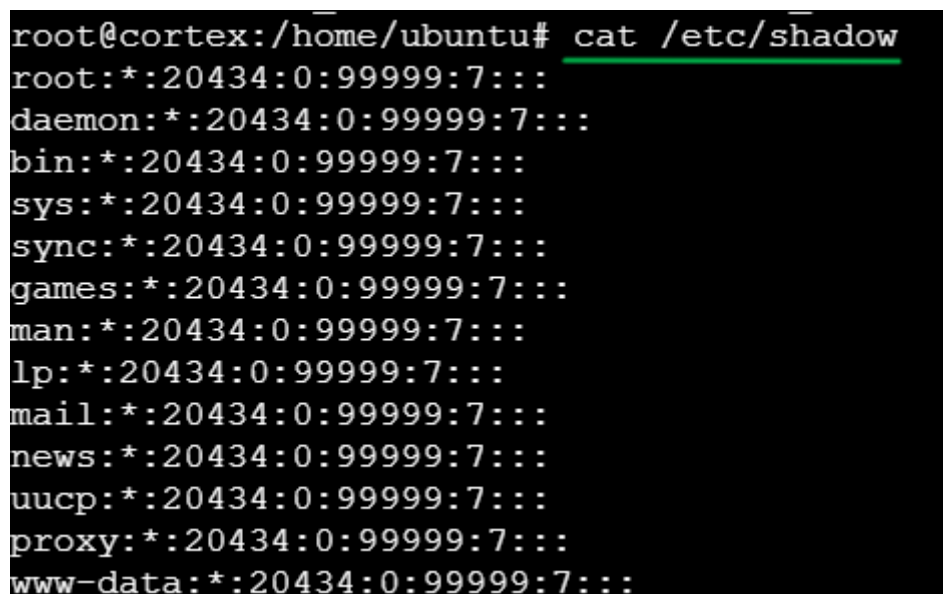
`cat /etc/shadow`

Purpose:

To simulate credential access behavior and observe how auditd logs the activity.

Audit logs confirmed:

- Command used
- User ID
- Session ID
- System call executed
- Target file



```

root@cortex:/home/ubuntu# cat /etc/shadow
root:*:20434:0:99999:7:::
daemon:*:20434:0:99999:7:::
bin:*:20434:0:99999:7:::
sys:*:20434:0:99999:7:::
sync:*:20434:0:99999:7:::
games:*:20434:0:99999:7:::
man:*:20434:0:99999:7:::
lp:*:20434:0:99999:7:::
mail:*:20434:0:99999:7:::
news:*:20434:0:99999:7:::
uucp:*:20434:0:99999:7:::
proxy:*:20434:0:99999:7:::
www-data:*:20434:0:99999:7:::

```

◆ Step 5 – Reviewing Audit Logs

Audit logs were examined in:

`/var/log/audit/audit.log`

Key fields observed:

- `audit.exe`
- `audit.command`
- `audit.execve.a0`
- `audit.file.name`
- `audit.key`

This demonstrated how auditd captures low-level forensic details.

```
time->Tue Feb 10 14:54:12 2026
type=PROCTITLE msg=audit(1770715452.355:501): proctitle=2F7362696E2F617564697463746C002D52002F6574632F61756469742E72756C6573
type=PATH msg=audit(1770715452.355:501): item=0 name="/etc/" inode=42 dev=ca:01 mode=040755 ouid=0 ogid=0 rdev=00:00 nametype=PARENT cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1770715452.355:501): cwd="/"
type=SOCKADDR msg=audit(1770715452.355:501): saddr=1000000000000000000000000000000000
type=SYSCALL msg=audit(1770715452.355:501): arch=c000003e syscall=44 success=yes exit=1080 a0=3 a1=7ffc45571cd0 a2=438 a3=0 items=1 ppid=10377 pid=10389 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none) ses=4294967295 comm="auditctl" exe="/usr/sbin/auditctl" subj=unconfined key=(null)
type=CONFIG_CHANGE msg=audit(1770715452.355:501): auid=4294967295 ses=4294967295 subj=unconfined op=add rule key="shadow access" list=4 res=1
```

◆ Step 6 – Integrating Auditd with Wazuh Agent

On the agent system, audit logs were configured inside:

`/var/ossec/etc/ossec.conf`

Block added:

```
<localfile>
  <log_format>audit</log_format>
  <location>/var/log/audit/audit.log</location>
</localfile>
```

Purpose:

To forward auditd logs to Wazuh Manager.

◆ Step 7 – Creating audit-keys CDB List (Manager)

File created:

`/var/ossec/etc/lists/audit-keys`

Purpose:

To map audit keys to meaningful categories such as:

- passwd
- shadow
- ssh
- history
- command

This allows Wazuh rules to match audit events effectively.

GNU nano 7.2

`/var/ossec/etc/lists/audit-keys`

```
exec_monitor
identity
privilege
ssh_config
cron
kernel_modules
passwd_access:passwd
shadow_access:shadow
history_access:history
ssh_access:ssh
audit-wazuh-w:write
audit-wazuh-c:command
audit-wazuh-r:read
audit-wazuh-a:attribute
audit-wazuh-x:execute
bash_history:bash
user_home_changes:home
root_home_changes:home
ssh_config_changes:ssh
```

◆ Step 8 – Creating suspicious-programs List

File created:

/var/ossec/etc/lists/suspicious-programs

entries:

```
GNU nano 7.2 /var/ossec/etc/lists/suspicious-programs *
nc:yellow
ncat:yellow
netcat:yellow
tcpdump:orange
strace:orange
gdb:red
```

Purpose:

To classify suspicious command executions.

This enables detection of attacker tools.

◆ Step 9 – Registering Lists in Wazuh

Inside:

/var/ossec/etc/ossec.conf

Added:

<list>etc/lists/audit-keys</list>

<list>etc/lists/suspicious-programs</list>

Purpose:

To make lists available for rule matching.

```
<!-- ===== -->
<!-- RULESETS -->
<!-- ===== -->
<ruleset>
  <decoder_dir>ruleset/decoders</decoder_dir>
  <rule_dir>ruleset/rules</rule_dir>

  <list>etc/lists/audit-keys</list>
  <list>etc/lists/suspicious-programs</list>
```

◆ Step 10 – Creating Custom Detection Rules

Custom rules were added inside:

/var/ossec/etc/rules/local_rules.xml

Examples:

- Detect access to shadow file
- Detect suspicious commands
- Detect credential search activity
- Map detections to MITRE ATT&CK

Purpose:

To generate high-fidelity alerts from auditd events.

```
<group name="cred_access,">
<!--Detect access to offline password storing files-->
  <rule id="100110" level="7">
    <if_sid>80700</if_sid>
    <list field="audit.key" lookup="match_key_value" check_value="passwd">etc/lists/audit-keys</list>
    <description>File access - The file $(audit.file.name) was accessed</description>
    <group>audit_command,</group>
    <mitre>
      <id>T1003.008</id>
    </mitre>
  </rule>

  <rule id="100120" level="10">
    <if_sid>80700</if_sid>
    <list field="audit.key" lookup="match_key_value" check_value="shadow">etc/lists/audit-keys</list>
    <description>Possible adversary activity - $(audit.file.name) was accessed</description>
    <group>audit_command,</group>
    <mitre>
      <id>T1003.008</id>
    </mitre>
  </rule>
```

```
<group name="audit">
  <rule id="100210" level="12">
    <if_sid>80792</if_sid>
    <list field="audit.command" lookup="match_key_value" check_value="red">etc/lists/suspicious-programs</list>
    <description>Audit: Highly Suspicious Command executed: $(audit.exe)</description>
    <group>audit_command,</group>
  </rule>
</group>
```

◆ Step 11 – Attack Simulation

Simulated activities:

- cat /etc/shadow

Purpose:

To validate detection capability.

```
root@cortex:/home/ubuntu# cat /etc/shadow
root:*:20434:0:99999:7:::
daemon:*:20434:0:99999:7:::
bin:*:20434:0:99999:7:::
sys:*:20434:0:99999:7:::
sync:*:20434:0:99999:7:::
games:*:20434:0:99999:7:::
man:*:20434:0:99999:7:::
lp:*:20434:0:99999:7:::
mail:*:20434:0:99999:7:::
```

◆ Step 12 – Alert Verification in Wazuh Dashboard

Alerts were observed in:

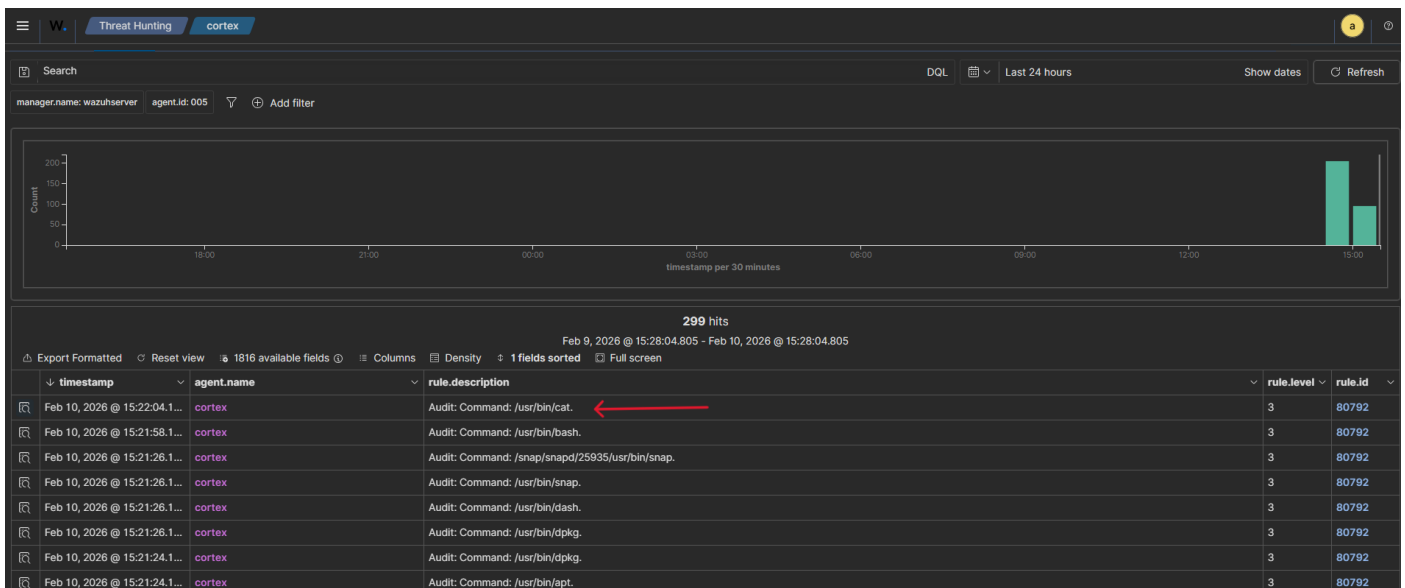
- Threat Hunting
- Discover

- Alert Details View

Key observed fields:

- agent.name
- audit.command
- audit.exe
- audit.file.name
- audit.key
- Rule ID

This demonstrated full visibility from kernel event to dashboard alert.



t _index	wazuh-alerts-4.x-2026.02.10
t agent.id	005
t agent.ip	
t agent.name	cortex
t data.audit.arch	c000003e
t data.audit.auid	1000
t data.audit.command	cat
t data.audit.cwd	/home/ubuntu
t data.audit.egid	0
t data.audit.euid	0
t data.audit.exe	/usr/bin/cat
t data.audit.execve.a0	cat
t data.audit.execve.a1	/etc/shadow
t data.audit.execve argc	2
t data.audit.exit	0
t data.audit.file.inode	1760
t data.audit.file.mode	0100755
t data.audit.file.name	/usr/bin/cat
t data.audit.fsgid	0
t data.audit.fsuid	0
t data.audit.gid	0
t data.audit.id	841
t data.audit.key	audit-wazuh-c
t data.audit.pid	23450
t data.audit.ppid	12974
t data.audit.session	6
t data.audit.sgid	0
t data.audit.success	yes

◆ Step 13 – Multiple Attack Testing

Additional tests performed:

- Credential hunting via grep
- SSH key inspection
- Bash history review
- Suspicious tool execution

Simulated activities:

- cat ~/.ssh/authorized_keys
- cat ~/.bash_history

- `grep -i login /etc/passwd`

All activities were logged and detected.

Purpose:

To validate detection capability.

```
root@cortex:/home/ubuntu# cat ~/.ssh/authorized_keys
no-port-forwarding,no-agent-forwarding,no-X11-forwarding,command=
AAAB3NzaC1yc2EAAAADAQABAAQDvIN9dWqhSjf4bokPm/U/dD3zjN/Q/foXY
```

```
root@cortex:/home/ubuntu# cat ~/.bash_history
docker --version
docker compose version
docker run hello-world
```

```
root@cortex:/home/ubuntu# grep -i login /etc/passwd
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
```

◆ Step 14 – Analyst View in Wazuh Discover

In Discover view:

- Full JSON event details visible
- User ID & session ID
- File accessed
- Executed binary
- System call metadata
- Audit key mapping
- Detection rule ID

This simulates real SOC investigation workflow.



Conclusion

This project demonstrates a complete implementation of:

- Kernel-level auditing using auditd
- SIEM integration with Wazuh
- Detection of Linux credential access attacks
- Real-time security monitoring
- Forensic-grade event visibility

The integration of auditd and Wazuh provides a powerful framework for proactive security monitoring, threat hunting, and compliance-driven logging.

This setup replicates a real-world SOC monitoring environment where analysts can detect, investigate, and respond to credential access attacks effectively.
